

# Data Structures and Algorithms — Lab 11

## Topic

Exploring Binary Trees and Binary Search Trees in C++: Structure, Traversal, and Manipulation

## Objective

1. Understand the fundamentals of binary trees and binary search trees.
2. Learn recursive implementation of insertion, traversal, and searching operations.
3. Distinguish between general binary trees and binary search trees (BST).
4. Practice real-world tree manipulation problems to build a strong foundation in hierarchical data structures.

## Outcomes

1. Implement binary trees and BSTs using recursive and iterative methods in C++.
2. Perform and explain various tree traversals: in-order, pre-order, post-order.
3. Search and insert elements in a BST.
4. Understand the application of tree data structures in organizing and accessing hierarchical data.

## Content

**The following sections will be covered during this lab session:**

1. **Introduction to Binary Trees:**
  - Binary Tree Node structure and manual tree creation.
  - Tree terminologies: root, leaf, depth, height, subtree.
2. **Tree Traversal:**
  - In-order: Left → Root → Right
  - Pre-order: Root → Left → Right
  - Post-order: Left → Right → Root
3. **Binary Search Tree:**
  - Rules and properties of BST.
  - Recursive insertion and searching.
  - Difference between Binary Tree and BST.

## Task Instructions

Students are required to complete the following tasks during lab time.

Create a private repository on your GitHub accounts. The name of the repository should be: **Lab-11-DSA**. All files should be uploaded to this repository; this includes the header and cpp files. Please note that the name of class-related files should be the same as the class name. We can name the main file Task\_1.cpp or Task.cpp if there is only 1 task. We recommend you work in .h files for classes only since we must work on templates.

### Task 1

Write a C++ program to manually construct a simple Binary Tree (not a Binary Search Tree) with at least 5 nodes. Each node should contain an integer value.

Once the tree is constructed, implement the following **recursive traversal functions**:

- **In-order Traversal**
- **Pre-order Traversal**
- **Post-order Traversal**

The program should output the elements of the tree in all three traversal orders.

### Task 2

Write a C++ program to implement a **Binary Search Tree (BST)** using insertion function. Your program should:

- Allow insertion of integer values into the BST based on the BST property.
- Implement in-order traversal function to display the elements of the BST in ascending order.

### Task 3

Extend your BST implementation in Task2 to include a **recursive search function**.

Your program should:

- Ask the user for a value to search.
- Return and print whether the value exists in the BST or not.

#### **Requirements:**

- The search must be implemented recursively.
- Return a message indicating whether the value was found.

#### Task 4

Write recursive functions to find the **minimum and maximum values** in a Binary Search Tree.

**Your program should:**

- Use recursion to reach the leftmost node for minimum.
- Use recursion to reach the rightmost node for maximum.

#### Task 5

Implement a **recursive deletion** function to remove a value from a Binary Search Tree.

Your program should:

- Prompt the user to enter a value to delete.
- Recursively remove the node and adjust the tree structure accordingly.
- Display the **in-order traversal** after deletion to verify the updated tree.

## Task 7- Magic Number

## BONUS-QUESTION

A wizard challenges you to find a **magic number** hidden in a sequence of numbers. The magic number is defined as the largest number in the sequence that is also equal to the sum of any subset of the remaining numbers in the sequence.

Given an array of integers, write a **recursive C++ function** that:

1. Finds the largest number in the array.
2. Checks recursively if this largest number can be represented as a sum of **any subset** (one or more elements) of the remaining numbers.
3. If yes, return that number (the magic number).
4. If no, remove that number and repeat the process with the next largest number.
5. If no such magic number exists, return -1.

### Constraints:

- You **must implement the subset sum check recursively** without loops.
- Use recursion to find and test the largest numbers as well.
- The array can contain positive integers only.

### Sample Output:

Input: [2, 3, 5, 8, 13]

- Largest number is 13.
- Can 13 be formed by sum of any subset of [2, 3, 5, 8]?
- Yes, because  $5 + 8 = 13$ .
- So, output: 13

*Good Luck! Stay focused, think in steps, and enjoy the logic!*